Module 9 Transmon Geometry Implementation Note

Detailed How-To Guide for `RadiationEffects_proj2`

Building, testing, inspecting, modifying, and handing off the reduced two-dimensional MATLAB geometry

---

Purpose and scope. This is the operational companion to the Module 9 physics note. It documents the geometry-only MATLAB layer: the code-ready transmon coordinates, the substrate mesh, named surface tags, automated validation, visualization, saved outputs, and the exact test procedure. It does *not* claim that Modules 2–6 or the Module 2 PINN already solve physics on this geometry. Those integrations are deliberately gated follow-on tasks.

Fastest correct start. Open MATLAB in the repository's `matlab/` directory, run `setup_project_paths`, run the master test `test_module9_transmon_geometry_2d`, then run `out = main_module9_transmon_geometry_2d();`. Passing the test suite and visually accepting both generated PNG files are separate requirements.

Contents

Symbols and acronyms used in this document

Symbol / acronym	Name	Meaning in this document
2D	Two-dimensional	The implemented cross-section uses coordinates (x, z).
FEM	Finite-element method	Later physics solvers will use the triangular substrate mesh.
JJ	Josephson junction	A one-micrometre sensitive interface tag, not a resolved oxide and not an electrical short.
x	Lateral coordinate	Horizontal coordinate, stored in metres.
z	Depth coordinate	Vertical coordinate; the substrate top is $z = 0$.
Ω_s	Substrate domain	Rectangle $[-1.5, 1.5]\,\mathrm{mm} \times [-0.5, 0]\,,\mathrm{mm}$.
Γ_k	Named boundary segment	An edge collection such as a pad, lead, bond pad, or backside sink.
h_x, h_z	Mesh spacings	Target lateral and depth increments before exact endpoint insertion.
CCW	Counter-clockwise	Required node ordering for every triangular element.
SI	International System of Units	All numerical geometry values are stored in metres.

1 What has been implemented

Module 9 currently supplies a reusable geometry description rather than a new field solver. The code performs four operations:

1. defines the material identities, dimensions, coordinates, mesh controls, visualization controls, and output controls;
2. creates a nonuniform, substrate-only triangular mesh whose lateral coordinate vector contains every required feature endpoint exactly;
3. converts the thin films and package features into named boundary-edge tags; and

4. validates the dimensions and topology, then creates a schematic view and a solver-facing mesh/tag view.

The implementation follows the corrected, code-ready convention in `docs/physics_notes/module9_transmon_geometry_microwave_package_fem_domains.pdf`. The large pads, junction leads, and JJ-sensitive interval form one geometrically connected centerline chain. The left and right solver electrode groups nevertheless remain disjoint because the JJ is excluded from both groups.

1.1 What is intentionally not implemented

- The curved wire-bond arcs are not mesh regions or tags. Their future electrical or thermal effects must enter at the bond-pad tags through a flux, source, Robin condition, or lumped link.
- The 100 nm aluminum, 80 nm trap, and 1 μm backside patch are not resolved as bulk elements. Their physical thicknesses are metadata attached to surface regions.
- No electrostatic, thermal, defect, carrier, or coupled solve is called by the Module 9 driver.
- The present code does not use MATLAB PDE Toolbox. It constructs the nodes, triangles, boundary edges, and tags directly with core MATLAB operations.
- There is no visual regression test. A person must inspect the two generated figures.

Central modeling rule. The presentation schematic exaggerates film height only to make nanometre films visible next to a 0.5 mm substrate. The physics mesh contains only the substrate and zero-thickness surface tags. Never pass the schematic display heights into a solver as physical layer thicknesses.

2 Files and call graph

2.1 Public entry points

File	Operational role
`matlab/main_module9_transmon_geometry_2d.m`	Main driver. Builds the geometry, throws on failed validation, makes the output directory, optionally generates two figures, saves a MAT-file and summary text, and returns the complete result.

File	Operational role
`matlab/tests/test_module9_transmon_geometry_2d.m`	Master Module 9 test runner. Calls the four focused regression tests in a fixed order. This is the normal test command.

2.2 Geometry implementation files

All reusable functions live in `matlab/src/module9_geometry/`.

Function file	Responsibility
`default_module9_transmon_geometry_2d.m`	Produces the baseline `params` structure in SI units. This is the authoritative executable parameter specification.
`build_module9_transmon_geometry_2d.m`	Orchestrates mesh creation, tagging, and non-throwing validation; returns the reusable `geom` structure.
`make_module9_transmon_mesh_2d.m`	Inserts required x anchors, creates refined x and z vectors, makes nodes and CCW triangles, and records complete boundary node/edge arrays.
`tag_module9_transmon_mesh_2d.m`	Selects boundary edges by midpoint, forms overlapping physical tags and disjoint solver electrode groups, and records the omitted wire arcs.
`validate_module9_transmon_geometry_2d.m`	Executes 27 aggregate checks and either returns a diagnostic report or throws one summarized error.
`plot_module9_transmon_geometry_2d.m`	Creates the presentation schematic with exaggerated film heights.
`plot_module9_transmon_mesh_tags_2d.m`	Creates the true-scale mesh/tag view and a central zoom that exposes the one-micrometre JJ tag.
`README.txt`	Records the strict integration gates that follow geometry acceptance.

2.3 Focused tests

The master test runner calls:

1. `test_module9_geometry_dimensions_2d.m`;

2. `test_module9_geometry_connectivity_2d.m`;

3. `test_module9_geometry_tags_2d.m`; and

4. `test_module9_geometry_no_wire_arc_2d.m`.

2.4 Execution flow

$$\text{parameters} \longrightarrow \text{substrate mesh} \longrightarrow \text{surface tags} \longrightarrow \text{validation} \longrightarrow \text{figures and saved geometry.} \tag{2.1}$$

The driver calls `build_module9_transmon_geometry_2d`, which attaches a non-throwing validation report. The driver then calls the validator again with `throwOnFailure=true`. Therefore no output is saved by the driver when a geometry invariant fails.

Direct-builder subtlety. Calling `geom = build_module9_transmon_geometry_2d(params)` directly does not throw when validation fails. Always inspect `geom.validation.passed`, or call `validate_module9_transmon_geometry_2d(geom,true)` before consuming a custom geometry.

3 Prerequisites and path verification

3.1 Software requirements

The geometry and tests require MATLAB with ordinary matrix, structure, plotting, and file-I/O functionality. No Deep Learning Toolbox and no PDE Toolbox are required. The plotting code prefers `exportgraphics` when available and falls back to `print`; it also uses `sgtitle` only when the installed release provides it.

MATLAB is required for the official runtime check. GNU Octave or an external script may help inspect numerical data, but neither replaces the MATLAB test result because figure handles, plotting behavior, export behavior, and MATLAB language details can differ.

3.2 Start from one unambiguous project copy

Extract one newly versioned project archive into a clean directory. Do not place two project copies on the same MATLAB path. Open MATLAB and change the Current Folder to the extracted `RadiationEffects_proj2/matlab/` directory.

Run:

```
setup_project_paths
```

```
which main_module9_transmon_geometry_2d -all
which default_module9_transmon_geometry_2d -all
which test_module9_transmon_geometry_2d -all
version
```

Each `which -all` result should resolve to the same extracted project. If an older project appears first, stop and remove that stale path before testing. The setup function adds `matlab/`, `matlab/cases/`, `matlab/tests/`, `matlab/pinn/`, and every directory below `matlab/src/`.

3.3 Check the baseline parameter function before a long inspection

```
params = default_module9_transmon_geometry_2d();
disp(params.schema)
disp(params.domain)
disp(params.modeling)
```

The schema should be `module9_transmon_geometry_2d_v1`; curved wires should be disabled; and the substrate-only bulk-mesh flag should be true.

4 Baseline geometry specification

4.1 Domain, layers, and materials

Quantity	Baseline value	Implementation meaning
Substrate x range	$[-1.5, +1.5]$ mm	Full lateral bulk domain.
Substrate z range	$[-0.5, 0]$ mm	Bottom to top of the bulk substrate.
Substrate area	$1.5\,\mathrm{mm}^2$	Cross-sectional area; not a three-dimensional volume.
Default substrate	High-resistivity silicon	Bulk material with relative permittivity 11.7.
Alternative metadata	Sapphire	Stored option with relative permittivity 9.4 but not an automatic selector.
Aluminum thickness	100 nm	Physical metadata for pads, leads, and bond pads.
Normal-trap thickness	80 nm	Stored above the aluminum film.

Quantity	Baseline value	Implementation meaning
Backside-patch thickness	$1\,\mu$m	Effective metadata for a later sheet/boundary model.
Lead out-of-plane width	$8\,\mu$m	Collapsed third-dimension metadata; not an x–z mesh length.

4.2 Code-ready feature intervals

All coordinates below are stored in metres; millimetres are shown for readability.

Feature	x interval (mm)	Length	Role
Left bond pad	$[-1.480, -1.400]$	$80\,\mu$m	Future package-interface BC location.
Left Al pad	$[-0.630, -0.030]$	$600\,\mu$m	Left electrode surface.
Left Al lead	$[-0.0405, -0.0005]$	$40\,\mu$m	Connects left pad to JJ; overlaps pad by $10.5\,\mu$m.
JJ-sensitive interval	$[-0.0005, +0.0005]$	$1\,\mu$m	Scoring/interface tag, excluded from electrode unions.
Right Al lead	$[+0.0005, +0.0405]$	$40\,\mu$m	Connects JJ to right pad; overlaps pad by $10.5\,\mu$m.
Right Al pad	$[+0.030, +0.630]$	$600\,\mu$m	Right electrode surface.
Normal-metal trap	$[+0.450, +0.550]$	$100\,\mu$m	Overlay on the right pad.
Right bond pad	$[+1.400, +1.480]$	$80\,\mu$m	Future package-interface BC location.
Backside sink	$[-0.150, +0.150]$	$300\,\mu$m	Bottom thermalization segment.

Feature	x interval (mm)	Length	Role
Pad-gap reference	$[-0.030, +0.030]$	$60\,\mu$m	Large-pad inner-edge window; centerline lead/JJ metal covers it.

The reference pad-gap window is not a separate exposed material region along this selected centerline. Consequently, `tags.reference.exposed_pad_gap.length` is expected to be zero.

5 Mesh construction and interpretation

5.1 Why the films are surface tags

Resolving a 100 nm film next to a 0.5 mm substrate creates a thickness ratio of

$$\frac{0.5\,\text{mm}}{100\,\text{nm}} = 5000. \tag{5.1}$$

A uniform element size small enough for the film would be wasteful and poorly conditioned for the present reduced model. The first implementation therefore meshes only Ω_s and retains physical layers as boundary metadata. Later weak-form terms can integrate over the named boundary edges.

5.2 Baseline mesh controls

Field	Value	Meaning
`params.mesh.bulkDx`	$25\,\mu$m	Requested lateral spacing in nominal bulk intervals.
`params.mesh.featureDx`	$10\,\mu$m	Requested spacing near pads, trap, and bond pads.
`params.mesh.centralDx`	$5\,\mu$m	Requested spacing near the central device. Feature endpoints remain exact even when a segment is shorter.
`params.mesh.bulkDz`	$25\,\mu$m	Requested depth spacing away from surfaces.
`params.mesh.surfaceDz`	$5\,\mu$m	Requested depth spacing near top and bottom surfaces.

Field	Value	Meaning
`params.mesh.surfaceRefinementDepth`	$50\,\mu$m	Nominal refined depth at each surface.
`params.mesh.alternateCellDiagonals`	`true`	Alternates triangle diagonals to reduce a single directional pattern.

Target spacings are upper targets inside intervals, not promises that every cell has exactly that width. The code uses `ceil` and `linspace`; short anchored intervals can be substantially smaller. Every domain, pad, lead, JJ, trap, bond-pad, pad-gap, and sink endpoint is inserted before interval filling.

5.3 Mesh arrays

Field	Meaning
`geom.mesh.nodes`	$N_n \times 2$ array of (x, z) node coordinates in metres.
`geom.mesh.elems`	$N_e \times 3$ array of one-based triangle node indices.
`geom.mesh.x`, `geom.mesh.z`	Structured coordinate vectors used to form the nodes.
`geom.mesh.nx`, `geom.mesh.nz`	Numbers of coordinates in each direction.
`geom.mesh.boundary.*`	Node IDs on each complete outer boundary.
`geom.mesh.boundaryEdges.*`	Two-node edge arrays for top, bottom, left, and right.
`geom.mesh.y`, `geom.mesh.ny`, `geom.mesh.Ly`	Compatibility aliases mapping the second generic coordinate to z.
`geom.mesh.origin`	Lower-left domain point $(-1.5, -0.5)$ mm.

Nodes are created with z varying fastest. Each structured quadrilateral becomes two three-node triangles. The validator requires every triangle's signed area to be positive, which verifies non-degeneracy and CCW ordering.

5.4 Probe the mesh directly

```
geom = build_module9_transmon_geometry_2d();
mesh = geom.mesh;

fprintf('nx=%d, nz=%d\n', mesh.nx, mesh.nz);
fprintf('nodes=%d, triangles=%d\n', ...
```

```
    size(mesh.nodes,1), size(mesh.elems,1));
fprintf('dx range = [%.3g, %.3g] um\n', ...
    min(diff(mesh.x))*1e6, max(diff(mesh.x))*1e6);
fprintf('dz range = [%.3g, %.3g] um\n', ...
    min(diff(mesh.z))*1e6, max(diff(mesh.z))*1e6);
```

Do not turn the default node or triangle count into a permanent scientific acceptance criterion. Counts legitimately change when mesh parameters change. Required coordinates, positive areas, correct tags, and convergence of the later physics quantity are the meaningful checks.

6 Boundary tags and solver semantics

Every tag stores its side name, boundary-local edge IDs, explicit global node pairs, unique node IDs, edge midpoints, and total physical length. The selection routine uses edge midpoints and a floating-point tolerance; exact endpoint insertion prevents ambiguous partial edges.

6.1 Physical and derived tag groups

Tag path	Meaning
`tags.boundary.left/right/top/bottom`	Complete outer substrate boundaries.
`tags.top.left_pad/right_pad`	Large aluminum pad intervals.
`tags.top.left_lead/right_lead`	Aluminum junction-lead intervals.
`tags.top.jj_sensitive`	Effective JJ-sensitive interface.
`tags.top.normal_trap`	Trap overlay; intentionally overlaps the right-pad tag.
`tags.top.left_bond_pad/right_bond_pad`	Future wire/package boundary-condition locations.
`tags.bottom.backside_sink`	Centered bottom thermalization segment.
`tags.electrode.left_electrode`	Union of left pad and left lead only.
`tags.electrode.right_electrode`	Union of right pad and right lead only.
`tags.derived.central_metal_chain`	Geometric union of pads, leads, and JJ. Do not use as a common-potential electrode.
`tags.derived.exposed_substrate_top`	Top-boundary edges not covered by primary aluminum features.
`tags.reference.pad_gap_window`	Nominal large-pad gap interval.

Tag path	Meaning
`tags.reference.exposed_pad_gap`	Empty in the current centerline convention.
`tags.omissions.curved_wire_arcs`	Machine-readable record that curved wires are absent.

Overlapping tags are intentional. A physical overlay such as the trap can share edges with the right pad, and a lead can overlap a pad. Do not assume that all tag edge-ID sets form a partition. The left and right *electrode groups*, however, must remain disjoint.

6.2 Inspect one tag

```
geom = build_module9_transmon_geometry_2d();
tag = geom.tags.top.jj_sensitive;

disp(tag)
disp(geom.mesh.nodes(tag.nodeIds,:)*1e3)  % coordinates in mm
fprintf('JJ tag length = %.6f um\n', tag.length*1e6);
```

The reported length should be $1\,\mu$m, and every associated node should lie on $z = 0$.

7 Exact run procedure

7.1 Acceptance sequence

Use this order for a fresh project copy:

1. Verify path resolution with `which -all`.
2. Create the default parameter structure and inspect its schema/modeling flags.
3. Run the master automated test suite.
4. Run the driver with figures enabled.
5. Inspect the Command Window validation count.
6. Inspect both PNG files manually.
7. Open the summary text and load the MAT-file.
8. Perform the direct probes in this note before modifying coordinates.

7.2 Run all Module 9 automated tests

From `RadiationEffects_proj2/matlab/`:

```
setup_project_paths
test_module9_transmon_geometry_2d
```

Expected pass messages are:

```
test_module9_geometry_dimensions_2d passed.
test_module9_geometry_connectivity_2d passed.
test_module9_geometry_tags_2d passed: 27 checks validated.
test_module9_geometry_no_wire_arc_2d passed.
All Module 9 reduced-geometry tests passed.
```

MATLAB stops at the first failed `assert`. Read that assertion message first; it normally identifies the broken invariant more precisely than the final aggregate validator would.

7.3 Run the driver and generate outputs

```
setup_project_paths
out = main_module9_transmon_geometry_2d();
```

The driver should print the number of nodes and triangles, report `27/27 passed`, restate that curved wire arcs are omitted, and display the absolute output directory.

7.4 Run without interactive windows

For a remote or headless MATLAB session:

```
setup_project_paths
params = default_module9_transmon_geometry_2d();
params.visualization.figureVisible = 'off';
out = main_module9_transmon_geometry_2d(params);
```

The PNGs are still written when `saveFigures=true`. If graphics are entirely unavailable, use `params.makePlots=false` for data-only debugging, but later rerun with plots because visual acceptance remains required.

7.5 Run an individual test during diagnosis

```
test_module9_geometry_connectivity_2d
```

Running one focused test is appropriate while repairing a specific failure. Before accepting the project, rerun the master suite so a local fix is not mistaken for complete acceptance.

8 What each test proves

8.1 Dimension test

`test_module9_geometry_dimensions_2d.m` verifies:

- 3 mm substrate width, 0.5 mm substrate thickness, and 1.5 mm^2 cross-sectional area;
- 100 nm Al, 80 nm trap, and 1 μm backside-patch metadata;
- correct vertical stacking metadata for the normal trap;
- 60 μm pad-gap reference width;
- 600 μm pad widths, 1 μm JJ width, 100 μm trap length, and 300 μm sink width.

It checks parameter/region metadata; it does not by itself prove that every boundary edge was tagged correctly.

8.2 Connectivity test

`test_module9_geometry_connectivity_2d.m` verifies the interval chain

$$\text{left pad} \rightarrow \text{left lead} \rightarrow \text{JJ} \rightarrow \text{right lead} \rightarrow \text{right pad}, \tag{8.1}$$

checks that the normal trap lies only on the right pad, confirms the JJ metadata still forbids an imposed short, and asserts that the two solver electrode edge-ID sets do not overlap.

This test proves interval contact or overlap in the reduced centerline geometry. It does not solve a circuit and does not prove Josephson behavior.

8.3 Tag test

`test_module9_geometry_tags_2d.m` first requests the full 27-check validator report, then independently compares primary tag lengths with analytical interval lengths. It also verifies that every top tag lies on $z = 0$, the sink lies on $z = -0.5$ mm, the pad-gap window is 60 μm, and the exposed centerline pad-gap length is zero.

8.4 No-wire-arc test

`test_module9_geometry_no_wire_arc_2d.m` makes the model reduction executable rather than merely descriptive. It requires the include flag to be false, the omission tag to report no wire arc, no wire-arc region field to exist, both bond-pad tags to remain nonempty, and the replacement description to refer to a boundary/lumped representation.

8.5 The 27 aggregate validator checks

The validator groups its checks as follows:

Group	Count	Checks
Domain dimensions	3	Width, thickness, and area.
Mesh structure	2	Every required x anchor is present; every triangle has positive signed area.
Central connectivity	4	Four pad/lead/JJ contact or overlap relations.
Trap placement	2	Contained in the right pad and does not bridge electrodes.
Primary tag lengths	9	Two pads, two leads, JJ, trap, two bond pads, and backside sink.
Headline dimensions	4	Pad-gap, JJ, backside-sink, and both bond-pad widths.
Stored layer ranges	1	Al, trap, and sink z-range metadata.
Reduced wiring decision	1	Curved wire arcs remain absent.
Tag indexing	1	Every primary tag edge ID lies within its named boundary array.
Total	27	All checks must pass.

To print failures and values without throwing:

```
geom = build_module9_transmon_geometry_2d();
report = validate_module9_transmon_geometry_2d(geom,false);
disp(struct2table(report.checks))
```

9 Visual verification

9.1 Presentation schematic

Inspect `matlab/outputs/module9_geometry/module9_transmon_geometry_2d.png`. Confirm:

- one 3 mm × 0.5 mm substrate rectangle;
- two large central Al pads and their narrow central leads;
- a visible central red JJ marker;
- the orange trap above the right pad only;
- one package bond pad near each lateral chip edge;
- one centered backside sink; and
- no curved wire arcs.

The visible film heights are deliberately false. They must not be used to measure Al, trap, or sink thickness.

9.2 Solver mesh and tag view

Inspect `matlab/outputs/module9_geometry/module9_transmon_mesh_tags_2d.png`. The upper panel must show the full true-scale substrate mesh, top surface tags, bond pads, and centered bottom sink. The lower panel must magnify the central pad/lead/JJ area so the 1 μm JJ is visible as a surface tag.

Check for missing colored segments, a JJ displaced from $x = 0$, a trap on the wrong pad, a sink on $z = 0$, or colors extending off the substrate boundary. Any of these is a failure even if the automated suite passes.

9.3 Why both images are required

The schematic answers "does the intended device layout look correct?" The mesh view answers "what geometry will a later FEM solver actually consume?" One cannot replace the other. The first uses visual-only film rectangles; the second uses exact mesh-edge segments.

10 Saved outputs and returned data

The default driver writes to `matlab/outputs/module9_geometry/`.

Output	Use
`module9_transmon_geometry_2d.png`	Presentation schematic with exaggerated vertical film heights.
`module9_transmon_mesh_tags_2d.png`	Solver-facing substrate mesh and zero-thickness tags.
`module9_transmon_geometry_2d.mat`	Reusable variable named `geometry`, containing parameters, mesh, tags, regions, materials, and validation.
`module9_transmon_geometry_summary.txt`	Human-readable dimensions, region intervals, mesh counts, validation checks, and modeling decisions.

The returned driver structure contains `out.geometry`, figure handles under `out.figures`, file paths under `out.files`, and `out.outputDir`.

File-path subtlety. The driver populates `out.files.*` paths even when a corresponding save option is false. A nonempty path field therefore does not prove that the file exists. Check `isfile(out.files.mat)` or `isfile(out.files.geometryPng)` when testing output creation.

10.1 Load and inspect the saved geometry

```
S = load(out.files.mat);
geometry = S.geometry;

assert(geometry.validation.passed)
disp(fieldnames(geometry))
disp(fieldnames(geometry.tags))
disp(geometry.tags.electrode)
```

The MAT-file intentionally excludes graphics handles. It is the object a future Module 2–6 integration should load or receive directly.

11 Independent probes before trusting or modifying the geometry

11.1 Verify every required coordinate is a mesh line

```
geom = out.geometry;
names = fieldnames(geom.regions);
for k = 1:numel(names)
    r = geom.regions.(names{k});
    err = max(arrayfun(@(q) min(abs(geom.mesh.x-q)), r.xRange));
    fprintf('%-18s max endpoint error = %.3e m\n', names{k}, err);
end
```

The endpoint error should be at roundoff level. If the pad-gap reference changes, repeat this check for `geom.reference.padGap.xRange`.

11.2 Verify electrode separation

```
leftIds = geom.tags.electrode.left_electrode.edgeIds;
rightIds = geom.tags.electrode.right_electrode.edgeIds;
jjIds = geom.tags.top.jj_sensitive.edgeIds;

assert(isempty(intersect(leftIds,rightIds)))
assert(isempty(intersect(leftIds,jjIds)))
assert(isempty(intersect(rightIds,jjIds)))
```

This is the solver-facing protection against accidentally applying one Dirichlet value through the JJ to both electrodes.

11.3 Verify tag locations and lengths

```
topNames = fieldnames(geom.tags.top);
for k = 1:numel(topNames)
    t = geom.tags.top.(topNames{k});
    fprintf('%-18s edges=%4d length=%9.3f um\n', ...
        topNames{k}, size(t.edges,1), t.length*1e6);
    assert(all(abs(geom.mesh.nodes(t.nodeIds,2)) < 1e-12))
end

sink = geom.tags.bottom.backside_sink;
assert(all(abs(geom.mesh.nodes(sink.nodeIds,2)+0.5e-3) < 1e-12))
```

11.4 Verify triangle orientation independently

```
p1 = geom.mesh.nodes(geom.mesh.elems(:,1),:);
p2 = geom.mesh.nodes(geom.mesh.elems(:,2),:);
p3 = geom.mesh.nodes(geom.mesh.elems(:,3),:);
A2 = (p2(:,1)-p1(:,1)).*(p3(:,2)-p1(:,2)) ...
   - (p3(:,1)-p1(:,1)).*(p2(:,2)-p1(:,2));
fprintf('minimum triangle area = %.6e m^2\n', min(A2)/2);
assert(all(A2>0))
```

12 Controlled parameter studies

12.1 Safe pattern

Always begin from a fresh default structure, make one coherent change, rebuild, validate with throwing enabled, and save to a separate output directory.

```
params = default_module9_transmon_geometry_2d();
params.outputDir = fullfile('outputs','module9_geometry_study_A');
params.visualization.figureVisible = 'on';

% Example mesh-only change; physical geometry is unchanged.
params.mesh.centralDx = 2.5e-6;

outStudy = main_module9_transmon_geometry_2d(params);
```

For mesh refinement, compare later physics quantities, not only the number of triangles. Geometry validation can pass on both a coarse and a fine mesh.

12.2 Changing a region interval

Region structures contain both `xRange` and the derived field `length`. If `xRange` changes, recompute `length`. If physical thickness changes, also update the appropriate global thickness metadata and region `zRange`. The code currently has no public "recompute all derived geometry fields" function.

```
params = default_module9_transmon_geometry_2d();
params.regions.normalTrap.xRange = [0.440e-3, 0.560e-3];
params.regions.normalTrap.length = diff(params.regions.normalTrap.xRange);
params.outputDir = fullfile('outputs','module9_geometry_wider_trap');
outStudy = main_module9_transmon_geometry_2d(params);
```

Changing only `xRange` and leaving stale `length` metadata should fail the tag-length validation.

That failure is protective, not a nuisance.

12.3 Changing the substrate material

The alternative sapphire structure is stored as metadata, but there is no string-valued material selector. A consistent change must assign the complete alternative structure:

```
params = default_module9_transmon_geometry_2d();
params.materials.substrate = ...
    params.materials.substrateOptions.sapphire;
```

This changes metadata only at the present geometry stage. A later FEM solver must actually read `geometry.materials.substrate` and assign the correct coefficients; Module 9 alone does not change any field equation.

12.4 Output and visualization controls

Parameter	Default	Effect
`params.makePlots`	`true`	Calls both plotting functions.
`params.saveFigures`	`true`	Exports both figures when plots are made.
`params.saveMat`	`true`	Saves the reusable geometry variable.
`params.saveSummary`	`true`	Writes the human-readable summary.
`params.visualization.figureVisible`	`'on'`	Opens interactive figure windows; use `'off'` for headless export.
`params.visualization.showLabels`	`true`	Adds schematic labels.
`params.visualization.imageResolution`	220	PNG export resolution in dots per inch.
`params.outputDir`	`outputs/module9_geometry`	Path relative to the MATLAB directory.

Use a distinct `outputDir` for each study. Otherwise deterministic filenames overwrite earlier outputs.

13 Troubleshooting

Symptom	Likely cause and response
Undefined function	Run `setup_project_paths`; use `which -all`; confirm MATLAB is in the correct extracted project's `matlab/` directory.
Wrong geometry appears	An older project copy is earlier on the path. Resolve all Module 9 functions with `which -all`; do not continue until they come from one tree.
Tag-length validation fails	A feature endpoint is missing, an interval changed without updating its derived `length`, or a boundary selector no longer captures the intended edges. Inspect `requestedXRange`, edge midpoints, and mesh anchors.
Positive-area check fails	Triangle ordering became clockwise or an x/z interval collapsed. Inspect the minimum signed area and the edited coordinate vectors.
Connectivity test fails	A pad/lead/JJ interval has a positive gap. Compare the four central `xRange` fields in millimetres.
Electrode tags overlap	The JJ was accidentally included in both electrode unions or a left/right interval crosses the center. Do not bypass this assertion.
Trap containment fails	The normal-trap interval extends off the right pad or toward the left electrode. Restore containment before using the geometry.
No-wire test fails	A curved wire was added as a region/tag, the omission flag changed, or the bond-pad replacement locations disappeared. Revisit the deliberate reduced-wiring model.
No PNG files	Check `makePlots`, `saveFigures`, output-directory permissions, and graphics availability. Remember that `out.files` paths exist even when saving is disabled.
Figures open but look vertically thick	Expected in the schematic. Use the mesh/tag figure for solver truth; do not interpret schematic height quantitatively.
JJ invisible in full-chip panel	Expected at true scale. Inspect the lower zoom panel or query `tags.top.jj_sensitive`.
Summary or MAT missing	Check `saveSummary` and `saveMat`; verify with `isfile`; confirm the driver reached output writing after validation.

Symptom	Likely cause and response
Output was overwritten	The same output directory and deterministic filenames were reused. Assign a unique `params.outputDir`.
Tests pass but a plot is wrong	The tests do not compare images. Treat manual visual inspection as a separate acceptance gate and diagnose the relevant plotting function.

14 What the current tests do not prove

Passing the master runner is necessary but not sufficient. The suite currently does not:

- execute the top-level driver or assert that PNG, MAT, and summary files were written;
- compare rendered images against approved reference images;
- solve any physics equation on the mesh;
- establish mesh convergence for potential, field, temperature, carrier density, or defect concentration;
- test sapphire coefficients in a downstream solver;
- test a resolved thin-film model;
- validate a three-dimensional package geometry; or
- establish that a lumped wire-bond model is quantitatively accurate.

Therefore the geometry acceptance record should contain the test transcript, two reviewed images, summary text, saved MAT-file, project version, and MATLAB version.

15 Code-reading order

For a reader studying the implementation line by line:

1. Read `default_module9_transmon_geometry_2d.m` to learn every physical and numerical assumption.
2. Read `main_module9_transmon_geometry_2d.m` to see the public workflow and output behavior.

3. Read `build_module9_transmon_geometry_2d.m` to see how mesh, tags, and validation are assembled.
4. Read `make_module9_transmon_mesh_2d.m` and sketch the node/triangle indexing.
5. Read `tag_module9_transmon_mesh_2d.m` to understand edge IDs, overlapping tags, unions, and omissions.
6. Read `validate_module9_transmon_geometry_2d.m` and count the 27 invariants.
7. Read both plotting functions, keeping display geometry separate from solver geometry.
8. Read the master test runner and then each focused test to understand the enforced contract.
9. Read `matlab/src/module9_geometry/README.txt` before changing any downstream module.

16 Strict follow-on integration plan

The accepted geometry becomes an upstream shared object. Integration should proceed through explicit gates so that a failure can be assigned to one layer.

16.1 Gate 1: accept Module 9 geometry

- Run the full automated suite in MATLAB.
- Run the driver and visually inspect both images.
- Confirm the MAT-file contains mesh, tags, regions, materials, and a passing validation report.
- Preserve the baseline output and test transcript before changing physics code.

16.2 Gate 2: connect Module 2 FEM

Preserve every legacy rectangular Module 2 case while adding a geometry selection and a transmon case. Required changes are:

Existing or new location	Required change
`default_module2_params.m`	Add legacy-rectangle versus Module 9 geometry selection and tagged-electrode BC settings.
`solve_poisson_defect_space_charge_2d.m`	Accept a supplied mesh/geometry instead of always constructing the old rectangle.

Existing or new location	Required change
`get_module2_dirichlet_nodes.m`	Consume `left_electrode` and `right_electrode` tags rather than whole chip sides.
`plot_module2_result_2d.m`	Overlay Module 9 tags and label coordinates (x, z).
`main_module2_electrostatics.m` and `matlab/cases/`	Add a named transmon trapped-charge case without changing existing cases.
`assemble_poisson_fem_2d.m`	Extend scalar permittivity only when nodal or elementwise material coefficients are actually needed.
`matlab/tests/`	Add Module 2-on-Module 9 tests for tag BCs, electrode separation, source sign, field direction, and legacy regression.

16.3 Gate 3: first tagged thermal validation

Use the bond-pad and backside-sink tags in an intentionally simple thermal test:

$$\text{bond-pad heat input} \longrightarrow \text{substrate heat spreading} \longrightarrow \text{backside thermal sink.} \tag{16.1}$$

This is the first clean test of segmented flux/source and sink boundary handling.

16.4 Gate 4: propagate one shared mesh through Modules 3–6

Module 3 should accept the external mesh; Module 4 should consume bond-pad and sink tags; Module 5 needs a genuine quasiparticle diffusion–reaction extension rather than relabeling electron–hole transport; and Module 6 should construct the Module 9 object once and pass the same mesh/tags to every participating module.

16.5 Gate 5: adapt the PINN last

The present Module 2 PINN assumes a simple rectangle, scalar permittivity, and boundary samples over complete sides. Adapt its sampling, boundary loss, input metadata, and reference data only after the tagged Module 2 FEM path has passed its own tests.

Immediate next task after geometry acceptance. Connect Module 2 FEM to the shared Module 9 mesh and the named left/right electrode tags while preserving all legacy rectangular tests. Do not begin the PINN conversion at the same time.

17 Minimal acceptance and handoff checklist

Before calling Module 9 geometry accepted, record:

- project archive version and exact extracted project path;
- MATLAB version and operating system;
- `which -all` results for the driver, default parameters, and master test;
- complete master-test transcript showing all focused tests and 27/27 validator checks;
- the two visually reviewed PNG files;
- the summary text file and reusable MAT-file;
- schema string and any parameter deviations from the baseline;
- node/triangle counts and minimum/maximum coordinate spacings;
- explicit confirmation that curved wire arcs were omitted;
- explicit confirmation that JJ edges are excluded from both electrode groups;
- any custom output directory used; and
- the next gate to be attempted, without mixing later gates into the accepted geometry baseline.

Final takeaway. The correct Module 9 procedure is path verification, automated invariant testing, driver execution, visual review of both geometry representations, saved-data inspection, and only then downstream integration. The master test runner is the automated contract; the two figures and MAT-file are the human and solver-facing evidence that completes that contract.